

GPT-2 para Machado de Assis

Gabriel Henrique do Nascimento Neres

I. DESCRIÇÃO DO PROBLEMA

Com o intuito de estudar sobre a geração de texto e o funcionamento de redes neurais, esse estudo visa o treinamento de uma pequena versão do modelo GPT-2 tendo como base para o treinamento os textos escritos pelo Machado de Assis. Nesse sentido, o modelo pretendido deverá ser capaz de gerar textos com características semelhantes aos textos do autor, além de buscar manter um texto consistente e coerente.

Além disso, para o enriquecimento do estudo, será feita uma nova implementação separada baseada no nanoGPT, a fim de observar perdas e/ou melhorias e ter uma melhor compreensão de cada uma das estruturas utilizadas. Nesse sentido, as mesmas configurações serão usados em ambos os modelos para decidir um melhor candidato para os testes seguintes.

Tendo decidido um melhor candidato, testes com múltiplas configurações poderão ser feitos para investigar as melhores configurações e possivelmente observar padrões de comportamento que auxiliem na escolha futura de parâmetros.

Desse modo, será possível definir uma melhor estratégia para a geração de modelos similares ao GPT-2 com uma base maior de maneira que tire melhor proveito dos dados obtidos.

II. METODOLOGIA

A. Preparação da base de dados

Para o desenvolvimento do modelo, a primeira parte será a obtenção da base de dados com os textos do Machado de Assis. Como os textos são de domínio público, existiam datasets já preparados com os textos, cada um em um formato diferente o que demandaria uma estratégia diferente para o seu uso. O dataset escolhido foi o apresentado na plataforma Kaggle [3], o qual apresenta diversos textos separados em arquivos distintos.

Primeiramente, foi feita uma limpeza dos dados, removendo a divisão de capítulos, reunindo novamente a mesma sentença em uma mesma linha, para ajudar na obtenção de contexto no treinamento, e a remoção de títulos, índices e demais elementos que fossem possíveis. Essa limpeza foi feita para remover elementos que não trariam grande informação na geração de padrões de escrita do autor, além de, possivelmente, atrapalhando o treinamento.

B. Preparação dos dados

Tendo o texto agrupado e preparado é preciso iniciar a preparação dessa informação que será utilizada como

entrada do modelo. Nesse sentido, a melhor forma para isso é a codificação da entrada para obter números para serem utilizados no modelo no lugar dos caracteres.

Existem diversas formas de codificação que podem ser utilizadas para a geração de diferentes modelos. Com o intuito de proporcionar uma melhor compreensão e comparação entre diferentes modelos, diferentes formas de codificação serão observadas.

1) *Hot-Encoding*: Ao utilizar o **Hot-Encoding**, cada um dos caracteres presentes seria representado por um vetor de 0s e um 1, o qual identificaria de maneira única cada caractere. Nesse sentido, foi adotada a adaptação de associar cada caractere a um inteiro, sendo que para a base obtida os números variam de 0 a 114 devido à presença de 115 caracteres únicos. Dessa forma o texto é transformado em uma lista de inteiros que será utilizada como a entrada do modelo.

2) *Embedding*: Os **Embeddings** se baseiam no conceito de associar cada entrada (um token ou um caractere) a um vetor de n dimensões que representa essa entrada no contexto de treinamento do modelo. Na abordagem utilizando diretamente os caracteres, cada caractere recebe um vetor com valores que indicam características com pesos que serão aprendidos durante o treinamento do modelo.

C. Bigram Model

Antes de ir ao modelo final alvo do estudo, que será o nanoGPT que irá emular o GPT-2 para uma pequena base de dados, é interessante passar por modelos menores para entender conceitos e analisar cada estrutura que será utilizada para o modelo final.

O primeiro modelo implementado é o Bigrama. Nele definimos a matriz que representará o Embedding de tamanho 115×115 que irá representar a interação de cada caracteres com cada um dos outros caracteres de maneira direta.

Na etapa **forward** do modelo, os batches do texto codificado são passados pela tabela de embedding e logo cada valor da lista de entrada aponta para um vetor de 115 posições terminando com uma matriz com dimensões de $B \times C \times 115$, B sendo o número de batches e C o comprimento do contexto (hiper parâmetros). Nesse mesmo método, quando o modelo também recebe o Y , o erro é calculado utilizando a função *Cross Entropy* adaptando o vetor X para o formato $B * C \times 115$ e o Y para $B * C$, pois assim cada valor de Y estará sendo comparado diretamente a um vetor de 115 elementos de X .

Para treinar o modelo, é utilizado o otimizador AdamW com um *Learning Rate* iniciando em 10^{-1} que decremente em um fator de 10x a cada 3 épocas. A cada iteração os gradientes do otimizador são zerados e re-calculados se baseando no erro calculado segundo o método anteriormente

citado, e na derivação do grafo do modelo obtido. Com os valores calculados o passo do otimizador é aplicado e passa para a próxima iteração até a iteração pretendida.

Ao testar o modelo, pode ser observado que o resultado ainda é bem ruim, não gerando palavras existentes nem mesmo a divisão entre as "palavras" na maioria dos casos.

D. Head of Self-Attention

Ao desenvolver o modelo GPT uma das estratégias utilizadas foi aumentar a área de atenção do modelo, ou seja, não limitar a análise aos caracteres diretamente subsequentes e passar a considerar a relação entre todo um trecho. Dessa forma, será construída uma relação mais complexa entre os termos e, consequentemente, com resultados mais relevantes. [5]

O primeiro elemento desse processo é mudar o embedding, mudando a matriz 115×115 pela matriz $115 \times E$, na qual E é o número de Embeddings que será um novo hiper parâmetro a ser definido. Com essa adaptação a estrutura da Head pode ser montada iniciando com a construção de 3 transformações lineares iguais do número de embeddings para o tamanho da head ($E \rightarrow H$).

Na etapa forward, a matriz de pesos é formada pela entrada X no formato (B, C, E) passa pela transformação linear, que é salva em duas variáveis no formato (B, C, H) , as multiplica entre si resultando na matriz no formato (B, C, C) e multiplica por $H^{-\frac{1}{2}}$ para controlar a variância nos valores e evitar o comportamento similar ao **Hot-Encoding**.

Sobre a matriz de peso é aplicada uma mascara triangular zerando os valores do triângulo superior, posteriormente aplicando o softmax e por ultimo o dropout para o treinamento. Dessa forma, o resultado final será obtido com a matriz de pesos multiplicada pela entrada x passando pela transformação linear $((B, C, C)X(B, C, H) \rightarrow (B, C, H))$.

Com essas alterações e o uso de múltiplas "cabeças" é possível observar uma pequena melhora, com o surgimento de algumas palavras conhecidas nos texto gerado. No entanto, a maior parte segue não tendo sentido e não forma nenhuma sentença completa.

E. nanoGPT

Ao reunir todos esse elementos, incluindo mais chamadas as cabeças de atenção, dropout, camadas feedforward e funções de ativação, temos a estrutura para uma versão em miniatura do GPT-2. Uma versão completa dessa redução pode ser encontrada no repositório [2] que é apresentado no vídeo [1].

O que será usado nesse estudo é baseado nesses exemplos após revisão e adaptação para maior facilidade na manipulação do modelo para os testes pretendidos.

F. Hiper parâmetros

Com o modelo finalizado, uma série de hiper parâmetros foram definidos para o funcionamento e configuração do modelo.

- **Nº de Batches (B)**: Define quantos trechos da base de testes serão selecionados por iteração do treinamento.
- **Tamanho de Contexto (C)**: Define o comprimento de cada trecho selecionado.
- **Nº de Embeddings (E)**: Determina o tamanho do vetor de embedding.
- **Numero de Heads (Hs)**: Define quantas cabeças de atenção serão utilizadas, e a partir dela determina o tamanho de cada head.
- **Nº de Camadas (blockLayers)**: Possibilita definir quantas camadas consecutivas de $\ln 1 - \zeta$ head + $x - \zeta \ln 2 - \zeta$ fwd + x serão utilizadas.
- **Dropout (dropout)**: Taxa de dropout utilizada no treinamento.
- **Bias**: Habilita ou desabilita o uso de biases no modelo.
- **Learning Rate (lr)**: Defini o valor máximo de taxa de aprendizado.
- **Weight Decay (wd)**: O valor a ser usado de decaimento nas camadas que permitem essa etapa.
- **Warmup Steps (warmup)**: Número de iterações que utilizarão valores de aprendizado que variam de $lr/warmup$ ao lr .
- **Learning Decay Step**: Determina a partir de que ponto do treinamento será utilizado o valor mínimo de aprendizado.
- **Min Learning Rate (minlr)**: Determina o valor mínimo de aprendizado.
- **Beta1 e Beta2 (betas)**: Betas utilizados no otimizador AdamW.

Com eles é possível ajustar e variar um pouco a estrutura e o funcionamento do modelo. Dessa forma, é possível investigar melhores estruturas para atender a base utilizada.

III. RESULTADOS

Após a preparação do máximo possível de dados para o aprendizado, foram obtidos um total de 7,2 milhões de caracteres para treinamento e um total de 2,3 milhões de caracteres para testes com um total de 137 caracteres distintos (que poderiam ser reduzidos com uma limpeza mais adequada de dados, como usando apenas um tipo de aspas simples e duplas, um tipo de travessão ...).

Realizando testes com o nanoGPT disponibilizado na internet e a versão simplificada desenvolvida nesse trabalho, foi observado uma grande diferença de desempenho. O tempo necessário entre as iterações eram 10 vezes maiores na versão deste trabalho, além de uma instabilidade maior

no treinamento, convergindo rapidamente para o ponto mínimo de erro e em seguida divergindo e aumentando drasticamente o erro. Dessa forma, a versão desenvolvida precisa de uma revisão mais profunda para obter resultados melhores, embora o texto final gerado seja muito similar ao gerado pelo nanoGPT.

Para comparação, a seguir podem ser observados os resultados obtidos com o nanoGPT após 10 mil iterações e a versão desenvolvida após 2 mil iterações. O primeiro apresenta um erro de 1,19 nos testes enquanto o segundo apresenta um erro de 2,5:

”Não acontece o que se pode repetir, dizia ele; toda a prima que me falou no pretendente do dia de moças.

— Vimos só que eu lhe pus na dirigida de Campos, com um sinal de pessoas para o vizinho.

Como assim dizia o amor, disse ela a uma certa intenção de castigo e um dia consentir os olhos do Castelo, alguns instantes e trabalhos de espírito.

— Se não serve de amor, amanhã ou chora.

— Casoiro...

— Mas as tuas qualidades eram um remédio do destino. Com efeito, não são coisas de santidade.”(nanoGPT)

”Não é encantado.

— Aqui havia ignior.

— A ernota nada.

Não nem é uma moça proversa: .

os bom; pensa começou crer em fácil, mas os que é um dia e outro ritor não atreve.

Se acudiu.

— Não o céu de um dia para em conveniente.

Quanto o primeiro é ao passador do país de madra, pura que lhe, malhei fazer a diferença, todo novo, levante, e tudo dedes, dez mais recondiva que eu precuso de um dogo, famo guerra fortunho; e saíram ainda sela impossível até caminha alguma vez.”(Versão desenvolvida)

A versão do modelo feita nesse trabalho está disponível no repositório do github por meio do link apresentada nas referências [4], inclusive com um Jupiter notebook demonstrando os passos feitos ao longo da metodologia.

Dado o melhor desempenho do NanoGPT, ele foi utilizado para aprofundar mais os testes desenvolvidos. Os valores utilizados para os hiper parâmetros foram:

- **Nº de Batches (B):** 64
- **Tamanho de Contexto (C):** 256
- **Nº de Embeddings (E):** 384
- **Numero de Heads (Hs):** 6
- **Nº de Camadas (blockLayers):** 6
- **Dropout (dropout):** 0.2
- **Bias:** Ativado
- **Learning Rate (lr):** 1e-3
- **Weight Decay (wd):** 1e-1

- **Warmup Steps (warmup):** 2000
- **Learning Decay Step:** 8000
- **Min Learning Rate (minlr):** 1e-5
- **Beta1 e Beta2 (betas):** 0.9 e 0.99

O menor taxa de erro na base de testes foi encontrada na iteração 20000 com erro no treino de 1,0833 e nos testes de 1,1830, de um total de 20 mil iterações realizadas, tendo uma melhoria de aproximadamente 0,007 após as 10 mil iterações. Dessa forma, podemos observar que não apresenta ganho em continuar o treinamento após 10 mil iterações e que o erro diminui de maneira mínima até 20 mil iterações com o modelo atual.

Com o intuito de analisar o impacto da variação dos parâmetros outros 5 valores foram testados, utilizando uma curva normal centrada nos valores apresentados anteriormente para gerar os novos valores. Os resultados obtidos podem ser observados no gráfico da imagem 1 e na tabela I.

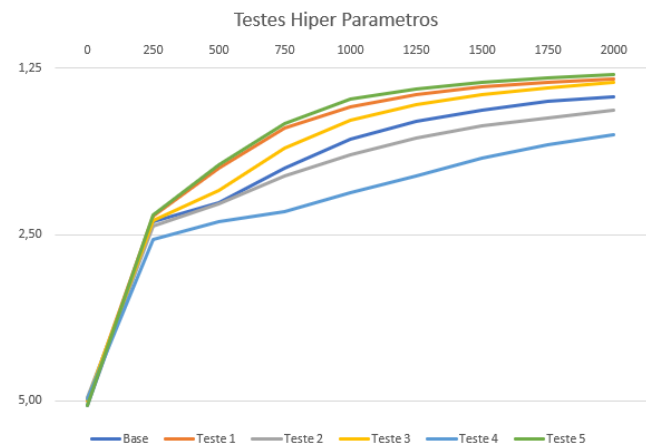


Figura 1. Gráfico comparativo entre os testes.

Tabela I
CONFIGURAÇÕES DOS PARÂMETROS UTILIZADAS NOS EXPERIMENTOS

Dados	batch_size	block_size	n_layer	n_head	n_embd
Base	64	256	6	6	384
Teste 1	88	233	6	9	684
Teste 2	61	127	11	9	261
Teste 3	89	293	8	6	402
Teste 4	26	266	11	10	170
Teste 5	89	268	6	11	781

Como pode ser observado, o teste com melhor resultado foi o de número 5 (que teve 44M de parâmetros) seguido pelo teste 1 (30M + de parâmetros). No entanto, o teste que mais chama a atenção foi o de número 3, que apesar de ter um número bem menor de parâmetros (15,6M) teve um resultado muito próximo, indicando um

bom equilíbrio no modelo gerado para tirar melhor proveito dos dados.

Outra observação importante foi o teste 4, que teve um valor significativamente menor no embedding e no batch size e o maior número de cabeças e camadas, e mesmo tendo o pior resultado, ainda não diferiu muito, indicando que ainda poderia apresentar um bom resultado com mais iterações mantendo um gasto computacional significativamente menor, dado que só possui 4M de parâmetros. Por esse motivo, uma nova execução até 10 mil iterações foi feita e o erro nos testes foi de 1.2804 no melhor caso, um erro aproximadamente 8% maior que o modelo base, porém com 60% do número de parâmetros.

Refazendo o teste 3 até 10 mil iterações, foi observado que o erro nos testes ultrapassou o modelo base na iteração 5500, e ao final das 10 mil iterações foi obtido um erro de 1.0078 nos testes e 1.1590, uma melhora de aproximadamente 2%.

Dessa forma, temos dados relevantes para apontar uma possível melhora na escolha dos parâmetros para o treinamento do modelo, como:

- Pelo teste 3, vemos que o aumento no número de camadas mantendo os demais parâmetros apresenta uma boa melhora sem aumentar muito o número de parâmetros.
- Pelo teste 2 vemos que o block size tem grande impacto no modelo, visto que sua redução associada ao aumento de camadas ainda levou a perdas em relação ao modelo base.
- Pelo teste 4 temos hipóteses, embora mais vagas dado a mudança expressiva, que é possível diminuir o batchSize.

O último ponto deste trabalho é refletir sobre como o conhecimento obtido a partir desse modelo podem ser estendidos para o universo sobre o autor. baseado na ideia de que o modelo de linguagem tem a capacidade de encontrar padrões no comportamento da linguagem, pode ser entendido que em um modelo mais complexo e com maior capacidade de explicabilidade, é esperado que seja possível compreender melhor as técnicas utilizadas pelo autor, vícios de linguagem, características do tempo vivido durante a escrita dos textos, entre outros.

REFERÊNCIAS

- [1] (3687) *Let's build GPT: from scratch, in code, spelled out.* - YouTube. URL: <https://www.youtube.com/watch?v=kCc8FmEb1nY&list=PLAqHlIrjxbuWI23v9cThsA9GvCAUhRvKZ&index=16> (acesso em 21/09/2026).
- [2] Andrej. *karpathy/nanoGPT*. original-date: 2022-12-28T00:51:12Z. 21 de set. de 2026. URL: <https://github.com/karpathy/nanoGPT> (acesso em 21/09/2026).
- [3] *Machado de Assis*. URL: <https://www.kaggle.com/datasets/luxedo/machado-de-assis> (acesso em 21/09/2026).
- [4] TheMasterGame0. *TheMasterGame0/GPT-2-Machado-Assis*. original-date: 2026-09-27T16:25:36Z. 27 de set. de 2026. URL: <https://github.com/TheMasterGame0/GPT-2-Machado-Assis> (acesso em 27/09/2026).
- [5] Ashish Vaswani et al. *Attention Is All You Need*. 2 de ago. de 2023. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762[cs.CL]. URL: <http://arxiv.org/abs/1706.03762> (acesso em 26/09/2026).